Séquence 1

COURS - Langage - Constructions élémentaires

Repère historique

En 1954, John Backus(*1924-2007*) présente le premier vrai langage de programmation tel qu'on l'entend aujourd'hui: le Fortran. Il mit au point vers la fin des années 50 le langage Algol. Il a, par la suite, proposé la *forme Backus-Naur* notation qui permet de décrire la *grammaire* des langages de programmation.

Programme et langage de programmation

Un **programme est un texte qui décrit un algorithme que l'on souhaite faire exécuter par une machine**. Ce texte est écrit dans un langage particulier, appelé **langage de programmation**. On le sauvegarde dans un fichier dont l'extension donne une indication sur le langage utilisé (`.py`, `.js`, `.c`, etc.)

Le principal langage utilisé en spécialité NSI est le langage **Python** (créé en 1991 par Guido Van Rossum) en version 3. C'est un langage de haut niveau (*d'abstraction*) et **interprété**, c'est-à-dire qu'il nécessite un autre programme appelé interpréteur qui va se charger d'exécuter ligne par ligne le programme de l'utilisateur.

Il existe bien évidemment d'autres langages de programmation. Il s'agit ici de présenter les **constructions élémentaires** que l'on retrouve dans la quasi totalité d'entre eux.

Les entrées/sorties

Dans un langage de programmation on appelle *entrées/sorties* des constructions permettant une communication avec l'utilisateur, donnant ainsi un aspect interactif au programme.
En python, les fonctions `print` et `input` remplissent ces taches.

```
# Exemples
print("Hello World!")
print("Python version 3.", "Environnement", "Jupyter Notebook 5.7")
print("Python version 3.", "Environnement", "Jupyter Notebook 5.7", sep=" :: ", end=" ")
print(" !!! Pas de retour à la ligne.")
```

```
Hello World!
Python version 3. Environnement Jupyter Notebook 5.7
Python version 3. :: Environnement :: Jupyter Notebook 5.7  !!! Pas de retour à la ligne.
```

Remarque

Les arguments optionnels comme `sep` ou `end` par exemple, permettent d'adapter l'affichage aux besoins.

Un effet (on dit aussi *effet de bord*) de la fonction `input` est d'interrompre le déroulement du programme et d'attendre que l'utilisateur tape quelque chose au clavier. Les caractères entrés sont généralement récupérés dans une **variable**.

```
niveau = input("Quel est votre niveau en programmation (novice/intermédiaire/avancé)? ")
moyenne = input("Quelle était votre moyenne annuelle en math ? ")
print("Vous avez répondu niveau: ", niveau,"avec ", moyenne, "en math!")
```

```
Quel est votre niveau en programmation (novice/intermédiaire/avancé)? avancé
Quelle était votre moyenne annuelle en math ? 20
Vous avez répondu niveau:  avancé avec  20 en math!
```

Remarques

- `input` **renvoie toujours une chaine de caractères** (même si l'utilisateur entre un nombre!);
- la syntaxe employée à l'intérieur de la fonction `print` bien que correcte, pourra être adaptée ultérieurement pour tenir compte des dernières évolutions du langage.

La déclaration-affectation

Pour pouvoir accéder aux données, un programme d'ordinateur fait abondamment usage de **variables**. En python, **une variable est un nom associé à un objet mémoire** (voir video pour les plus curieux).

Contrairement à certains langages, les variables n'ont pas besoin d'être déclarées au préalable, en python. La **déclaration et l'affectation de variables** peuvent se faire en une seule instruction. En python cette opération est réalisée selon le schéma `nom_variable = valeur`.

Exemples

```
n = 7
msg = "Affectation"
pi = 3.14
```

Remarques

- le nom de variable doit commencer par une lettre et ne pas être un mot réservé (voir liste).
- Le **type** de l'objet référencé par une variable est déterminé lors de l'affectation, de manière tout à fait transparente. Dans l'exemple précédent, `n` désigne un entier (`int`), `msg` désigne une chaîne de caractères (`str`), `pi` désigne un nombre décimal (`float`).

Séquence et bloc d'instructions

On peut définir une séquence d'instructions comme un ensemble de deux instructions exécutées l'une **à la suite** de l'autre (l'ordre est important!). Par exemple, les lignes

```
x = x + 1
x = x * 2
```

constituent une séquence d'instructions (*valide à condition que la variable x ait été affectée au préalable*). Nos programmes comporteront généralement des séquences de plus de deux instructions, on parlera alors de **bloc d'instructions**.
En python, un bloc d'instructions est repéré par son **indentation**, c'est-à-dire le décalage par rapport au bloc précédent (*typiquement un multiple de 4 espaces*).

Exemple

La séquence d'instructions ci-dessous permet de calculer et d'afficher l'aire S d'un disque de rayon r (rappel: $S = \pi \times r^2$).

```
PI = 3.14
r = 5
s = PI * r**2
print(s)
```

```
78.5
```

Remarque

Les instructions précédentes forment un seul bloc d'instructions (elles sont toutes alignées sur la même colonne).

Le test simple

Cette construction permet d'exécuter une instruction ou un bloc d'instructions uniquement si une condition est vérifiée. En python, la syntaxe est la suivante:

```
if condition:
    bloc_instructions
```

Exemple

La construction `x % 2 == 0` (**comparaison** du reste de la division euclidienne de x par 2 avec zéro) permet de tester si x est pair.

```
n = 50
if n % 2 == 0:
    print(n, "est pair!")
    print("Sa moitié vaut ", n / 2)
```

```
50 est pair!
Sa moitié vaut  25.0
```

Pour identifier sans ambiguités les instructions appartenant au bloc vérifiant la condition, on **DOIT LES INDENTER** en python.

La condition est souvent exprimée avec des **opérateurs de comparaisons**. Voici quelques exemples:

```
x == y # x est égal à y
x != y # x est différent de y
x > y # x est plus grand que y
x < y # x est plus petit que y
x >= y # x est plus grand que, ou égal à y
x <= y # x est plus petit que, ou égal à y
```

Le test avec alternative

Dans un test avec alternative, on ajoute un bloc à traiter lorsque **la condition n'est pas vérifiée**. La syntaxe est la suivante:

```
if condition:
    bloc_instructions_SI
else:
    bloc_instructions_SINON
```

Exemple

```
# Les test alternatif en python
n = 501
if n % 2 == 0:
    print(n, "est pair!")
    print("Sa moitié vaut ", n / 2)
else:
    print(n, " est impair")
```

```
501  est impair
```

La boucle conditionnelle, non bornée

Une **boucle conditionnelle**, appelé encore **boucle while** (*en anglais*) ou **boucle tant que** (*en français*) exécute un bloc d'instructions tant qu'une condition est vérifiée.

En python, la boucle conditionnelle a la forme suivante:

```
while condition:
    bloc_instructions
```

Remarques

- la condition s'exprime généralement avec les opérateurs de comparaisons vus plus haut;
- le bloc d'instructions est indenté.

Exemple

```
tab = 8
i = 1
while i < 10: # la condition
    print(i, " x ", tab, " = ", i*tab)
    i = i + 1
```

```
1  x  8  =  8
2  x  8  =  16
3  x  8  =  24
4  x  8  =  32
5  x  8  =  40
6  x  8  =  48
7  x  8  =  56
8  x  8  =  64
9  x  8  =  72
```

ATTENTION

Il **faut que la condition ne soit plus vérifiée** à un moment donné pour assurer la **terminaison** de la boucle et éviter le bouclage infini. Ce point sera détaillé dans le cours d'algorithmique.
Dans le cas précédent, la condition est: $i < 10$. Or, i est un entier qui augmente de 1 à chaque tour de boucle. Lorsqu'on aura atteint $i = 10$, la condition ne sera plus vérifiée et on sortira de la boucle.

La boucle inconditionnelle ou *bornée*

Lorsque le **nombre d'itérations est connu d'avance** la boucle devient **inconditionnelle**. On l'appelle encore boucle bornée ou **boucle pour** en français (*for loop* en anglais).
En python, la syntaxe de la boucle `for` est la suivante:

```
for variable_compteur in sequence:
    bloc_instructions
```

Remarques

- `sequence` est une suite d'objets, comme des entiers, des caractères d'une chaine, etc, (d'autres séquences seront vus dans les prochains cours); typiquement une séquence d'entiers est obtenue avec l'objet `range(a, b)` qui fourni les entiers allant de a à $b - 1$.
- Le bloc d'instructions doit être indenté.

Exemple

```
tab = 10

for i in range(1, tab): # i est la variable compteur, range(1, tab) fourni la sequence
d'entiers
    print(i, " x ", tab, " = ", i*tab)
```

```
1  x  10  =  10
2  x  10  =  20
3  x  10  =  30
4  x  10  =  40
5  x  10  =  50
6  x  10  =  60
7  x  10  =  70
8  x  10  =  80
9  x  10  =  90
```

Diversité et unité des langages

Les langages de programmations sont très nombreux. Ils peuvent être classés suivant leur *style* ou **paradigme** (façon de modéliser les problèmes et les résoudre).

Plusieurs paradigmes (orienté objet ou fonctionnel) seront abordés en terminale. En 1re, on peut se limiter au style **impératif**, dans lequel les problèmes sont résolus de manière séquentielle, en procédant à des modifications de la mémoire. Il est fait usage de *variables*, de *boucles*, etc.

Passer d'un langage à un autre peut être facile à l'intérieur d'un même paradigme car les constructions sont souvent assez proches.

On peut aussi distinguer les langages suivant le fait qu'ils soient:

- interprété[1] ou compilé[2];
- à typage dynamique[3] ou statique[4];

Exemples

Le test alternatif en Javascript

```javascript
const N = 50;

if (N % 2 === 0) {
    console.log(N + " est pair");
    console.log("Sa moitié vaut " + N/2);
} else {
    console.log(N + " est impair");
}
```

Le test alternatif en C

```c
#include <stdio.h>

int main(void) {
    int N = 50;

    if ((N % 2) == 0){
        printf("%d est pair\n", N);
        printf("Sa moitié vaut %d\n", N/2);
```

```
        }
    else {
        printf("%d est impair\n", N);
        }
    return 0;
}
```

---

1. un programme annexe lit le code source et exécute les instructions; c'est le cas du langage Python (l'interpréteur le plus courant s'appelle CPython). ↩
2. un programme annexe transforme le code source en langage machine (une fois) avant son exécution. ↩
3. une variable n'a pas besoin d'être déclarée et peut changer de type au cours du programme (exemple en python). ↩
4. les variables doivent être déclarées et ne peuvent changer de type. ↩